

FOOD DELIVERY SYSTEM

Software Requirements Specification + SOLID & Design Pattern Practice Blueprint

Purpose: Build a realistic object-oriented food-delivery system and use the requirements to discover responsibilities, abstractions, relationships, and suitable design patterns. The document deliberately avoids giving the final class diagram or forcing a pattern into every feature.

How to use this document

- Read the requirements first and identify domain objects and behaviors.
- Create your first UML/domain model before thinking about patterns.
- Apply SRP, OCP, LSP, ISP, and DIP to improve the design.
- Only then select patterns where the design has a genuine recurring problem.
- For every pattern, be able to explain the problem it solves, why it fits, and what trade-off it introduces.

Important: SOLID principles are design principles; design patterns are reusable solutions. Do not add a pattern merely because the project contains a place where it can be demonstrated.

1. SYSTEM SCOPE

The system is a multi-role food-delivery platform. Customers discover restaurants, manage carts, place and pay for orders, track deliveries, receive notifications, and submit reviews. Restaurants manage menus and process orders. Delivery partners handle assigned deliveries. Administrators manage platform entities, complaints, and reporting.

Primary roles

- **Customer:** searches restaurants, manages addresses, creates carts, places orders, pays, tracks orders, cancels when allowed, and reviews completed orders.
- **Restaurant Owner/Staff:** manages restaurant information and menu, accepts/rejects orders, and updates preparation status.
- **Delivery Partner:** maintains availability/location, accepts deliveries, picks up orders, and completes delivery.
- **Administrator:** approves/suspends restaurants and delivery partners, manages platform offers, complaints, and reports.
- **External providers:** payment, notification, and potentially search/location services.

2. FUNCTIONAL REQUIREMENTS

2.1 User & Authentication

- Register with name, email, phone, and password.
- Log in, log out, and maintain authenticated sessions.
- Update profile and change password.
- Support roles: CUSTOMER, RESTAURANT_OWNER, RESTAURANT_STAFF, DELIVERY_PARTNER, ADMIN.
- Authorization must prevent a user from performing operations outside their role or accessing another customer's private data.

2.2 Address Management

- Customer can add, update, delete, and select a default delivery address.
- Address contains house/building, street, city, state, postal code, latitude, longitude, and type (HOME, WORK, OTHER).

- An order stores the delivery address information needed for that order so later profile/address changes do not silently change historical orders.

2.3 Restaurant Discovery

- Search by restaurant name, food name, cuisine, and location.
- Filter by rating, price range, distance, and availability.
- Restaurant details include name, description, address, cuisine, rating, operating hours, delivery fee information, and estimated delivery time.
- Restaurant operational states: OPEN, CLOSED, TEMPORARILY_UNAVAILABLE.
- A customer cannot place a new order when the restaurant cannot accept orders.

2.4 Menu & Food Items

- Restaurant has a menu containing categories and food items.
- Food item contains ID, name, description, price, category, image reference, availability, and vegetarian/non-vegetarian indicator.
- Restaurant staff can add, update, remove, and enable/disable food items and categories.
- Unavailable food cannot be added to a cart.

2.5 Cart

- A customer can maintain a cart containing food items from one restaurant.
- Cart supports add, remove, update quantity, clear, and total calculation.
- Totals may include item subtotal, discount, tax, delivery fee, and final amount.
- Adding an item from another restaurant must trigger a business decision: reject the addition or replace/clear the existing cart.
- Cart quantity must be greater than zero.

2.6 Offers & Discounts

- Support percentage discounts, fixed discounts, minimum-order discounts, restaurant-specific offers, and first-order offers.
- Before applying a discount, the system validates eligibility and calculates the discount amount.
- New discount rules should be addable without rewriting unrelated order/cart logic.

2.7 Checkout & Order Creation

- Checkout shows restaurant, items, quantities, prices, delivery address, subtotal, discount, tax, delivery fee, payment method, and final amount.
- Placing an order creates an immutable snapshot of ordered item prices and relevant order details.
- Order contains order ID, customer, restaurant, delivery address, order items, quantities, prices, totals, payment status, order status, and timestamps.
- The system must avoid creating duplicate orders when the same checkout request is accidentally submitted more than once.

2.8 Order Lifecycle

- Normal lifecycle: PLACED → CONFIRMED → PREPARING → READY_FOR_PICKUP → PICKED_UP → OUT_FOR_DELIVERY → DELIVERED.
- An order may become CANCELLED when business rules allow it.
- Invalid transitions must be rejected. For example, DELIVERED must not return to PREPARING.
- Restaurant staff update preparation-related states; delivery partners update pickup/delivery-related states; customers can view status.

2.9 Restaurant Order Processing

- Restaurant receives newly placed orders.
- Authorized restaurant staff can accept or reject an order.
- Accepted orders move through preparation and ready-for-pickup states.
- Restaurant rejection must trigger appropriate customer/payment handling.

2.10 Delivery Assignment & Tracking

- Delivery partner has ID, name, phone, vehicle details, location, and availability state: AVAILABLE, BUSY, OFFLINE.
- The platform assigns a suitable delivery partner after the order is ready/eligible for delivery assignment.
- Assignment may consider availability, restaurant distance, customer distance, and current workload.
- Customers can see delivery partner information and delivery progress when appropriate.
- The assignment strategy must be replaceable so alternative algorithms can be introduced later.

2.11 Payments

- Initially support CARD, UPI, and CASH_ON_DELIVERY; design should allow future providers/methods.
- Payment records transaction ID, order ID, amount, method, status, and timestamp.
- Payment states: PENDING, SUCCESS, FAILED, REFUNDED.
- Failed payments can be retried where allowed.
- The system must prevent accidental double charging for the same logical transaction.

2.12 Cancellation & Refunds

- Cancellation eligibility depends on order state and business rules.
- Examples: early states may be cancellable; PICKED_UP and DELIVERED should not be cancellable by the normal customer flow.
- If money was already captured and cancellation requires a refund, a refund operation is initiated.
- Refund contains refund ID, transaction ID, amount, reason, status, and timestamp.

2.13 Notifications

- Notify users for order placement, confirmation, rejection, preparation, readiness, driver assignment, pickup, delivery, payment failure, and refund events.
- Initially support email, SMS, and push notification.
- New notification channels should be addable without rewriting order business rules.

2.14 Reviews & Ratings

- After successful delivery, customer can rate the restaurant and delivery partner and optionally add food comments.
- Ratings are 1–5 stars.
- A customer cannot review an undelivered order or submit multiple reviews for the same order where the business rule permits only one review.
- Restaurant rating is derived from eligible customer reviews.

2.15 Administration

- Approve/reject/suspend restaurants.
- Manage or suspend delivery partners.
- View customers, restaurants, orders, payments, refunds, and complaints.
- Create/manage platform-wide offers.

- Access operational and revenue reports.

2.16 Complaints

- Customer can create a complaint linked to an order.
- Categories may include missing food, wrong food, damaged food, late delivery, payment problem, and refund problem.
- Complaint contains ID, customer, order, category, description, created time, and status.
- Statuses: OPEN, IN_PROGRESS, RESOLVED, CLOSED.
- Authorized support/admin users can update complaint status.

2.17 Reporting

- Provide basic order, revenue, customer, and restaurant reports.
- Examples: total/completed/cancelled orders, daily/monthly revenue, active customers, active/suspended restaurants.

3. BUSINESS RULES & INVARIANTS

- **1.** Only authorized roles can perform role-specific operations.
- **2.** A cart cannot contain items from multiple restaurants.
- **3.** Unavailable food cannot be added to a cart.
- **4.** An order preserves the item prices used at checkout.
- **5.** Order state transitions must follow the allowed lifecycle.
- **6.** Cancellation must obey the order's current state and business policy.
- **7.** Only eligible completed/delivered orders can be reviewed.
- **8.** A review cannot be duplicated for the same order when the one-review rule applies.
- **9.** Payment attempts must be idempotent enough to avoid accidental duplicate charging/order creation.
- **10.** Restaurant must be open/accepting orders before a new order can be placed.
- **11.** Historical orders must remain readable even when menu items, prices, addresses, or restaurant data change later.
- **12.** Business logic should not be tightly coupled to a specific payment gateway, notification provider, search implementation, or delivery-assignment algorithm.

4. NON-FUNCTIONAL REQUIREMENTS

- **Performance** — Restaurant and food searches should respond quickly; common read operations should be efficient.
- **Scalability** — The architecture should allow growth in customers, restaurants, orders, and delivery partners without a complete rewrite.
- **Maintainability** — Responsibilities should be clear and changes in one feature should minimize unrelated modifications.
- **Extensibility** — New payment methods, notification channels, discount rules, delivery strategies, search implementations, tax rules, and fee policies should be reasonably easy to add.
- **Reliability** — Avoid duplicate orders/payments, invalid state transitions, inconsistent totals, and lost critical state changes.
- **Security** — Protect credentials and payment information; enforce authentication/authorization and data isolation.

5. DOMAIN MODEL — WHAT YOU SHOULD DISCOVER

The following are candidate domain concepts, not a final class diagram. Validate each against the requirements. Some may become entities, value objects, services, interfaces, enums, or modules depending on your design.

Candidate concepts	Questions to ask
User / Customer / RestaurantOwner / RestaurantStaff / DeliveryPartner / Admin	Who owns it? What data must persist? What invariants does it protect?
Restaurant / Menu / MenuCategory / FoodItem	Is it a real domain object or just a collection of data?
Address / Location	Does it have behavior, or should behavior live elsewhere?
Cart / CartItem	Does it have multiple implementations?
Order / OrderItem	Can its rules change independently?
Payment / PaymentAttempt / Refund	Does it need an interface, or would an interface add unnecessary complexity?
Discount / Offer	Who owns it? What data must persist? What invariants does it protect?
Delivery / DeliveryAssignment	Is it a real domain object or just a collection of data?
Notification	Does it have behavior, or should behavior live elsewhere?

Candidate concepts	Questions to ask
Review / Rating	Does it have multiple implementations?
Complaint	Can its rules change independently?
Report	Does it need an interface, or would an interface add unnecessary complexity?

6. SOLID REVIEW CHECKLIST

S — SRP

For each class: what are its reasons to change? If unrelated business rules change the same class, separate responsibilities.

O — OCP

Identify variation points such as payment, discount, notification, search, delivery assignment, tax, and fee calculation. Ask whether adding a new variant requires editing stable business logic.

L — LSP

For every interface/base type, define its behavioral contract. Every implementation must be safely usable wherever that abstraction is expected; no implementation should need to break the contract or throw merely because the abstraction promised something it cannot do.

I — ISP

Check interfaces for clients/implementations that depend on methods they do not need. Split broad contracts when separate clients require separate capabilities.

D — DIP

High-level business policies should not be tightly coupled to concrete infrastructure details. Both should depend on suitable abstractions.

7. NATURAL DESIGN-PATTERN OPPORTUNITIES

These are deliberately framed as problems to investigate, not mandatory answers. Try to design without patterns first; then see which pattern, if any, makes the design clearer.

Pattern	Problem it solves	Where to investigate
Strategy	Multiple interchangeable algorithms/rules	Delivery assignment, discount calculation, delivery fee calculation, tax calculation, restaurant/search ranking.
Factory Method / Simple Factory / Abstract Factory	Object creation varies by type or provider	Payment method/provider objects, notification channel objects, potentially report/export components. Choose the simplest appropriate factory approach.
State	Object behavior and allowed transitions depend strongly on current state	Order lifecycle and possibly delivery/payment lifecycle.
Observer / Publish-Subscribe	One event should notify multiple independent consumers	Order-status events triggering notifications, analytics, audit, or other subscribers.
Adapter	A third-party API has an incompatible interface	Payment gateway, SMS/email/push provider, map/geocoding service.
Facade	Client needs a simple entry point over a complex subsystem	Checkout/order placement orchestration over cart validation, pricing, payment, order creation, and notification.
Template Method	Several workflows share a stable skeleton with controlled variation	Potentially reporting or provider workflows, only if inheritance genuinely fits.
Builder	Construction has many optional/validated parts	Potentially complex immutable request/order objects, if constructors become unwieldy.

Pattern	Problem it solves	Where to investigate
Command	Represent an operation as an object	Potentially admin actions, retryable operations, queues, or audit-friendly actions.
Decorator	Add responsibilities dynamically around an object	Potentially layered pricing/notification/logging behavior, if a genuine need appears.

8. USE-CASE MAP

Actor	Major use cases
Customer	Register/login; manage profile; manage addresses; search restaurants/food; view menu; manage cart; apply offer; checkout; pay; view/cancel orders; track delivery; review order; create complaint; reorder.
Restaurant Staff	Manage restaurant/menu/categories/items; view incoming orders; accept/reject; update preparation status; view order history.
Delivery Partner	Set availability; receive assignment; accept/reject where policy allows; view delivery; update pickup/out-for-delivery/delivered states; update location.
Admin	Approve/suspend restaurants; manage delivery partners; manage offers; view platform data; resolve complaints; view reports.
External Provider	Authorize/capture/refund payment; send notification; optionally provide search/location functionality.

9. CORE WORKFLOWS TO DESIGN

Place Order

Customer → select restaurant → add items → validate cart → calculate subtotal/discount/tax/fee → select address/payment → create order safely → process payment → confirm/update order → notify relevant parties.

Restaurant Processing

Restaurant receives order → accept/reject → preparation → ready for pickup → delivery assignment becomes eligible.

Delivery

Find eligible delivery partners → select/assign according to strategy → partner accepts → pickup → out for delivery → delivered → order completion.

Cancel & Refund

Receive cancellation request → check order state/rules → cancel if allowed → determine payment/refund requirement → initiate refund if needed → notify customer.

Review

Verify order belongs to customer → verify delivered state → verify review eligibility → save rating/review → update/derive restaurant/partner rating.

10. UML/DESIGN DELIVERABLES

- Use Case Diagram — actors and major system interactions.
- Class Diagram — entities, interfaces, services, repositories, and relationships.
- Sequence Diagram — at minimum: Place Order, Payment, Delivery Assignment, Cancel/Refund.
- State Machine Diagram — Order lifecycle and valid transitions.
- Component Diagram — presentation/application/domain/infrastructure boundaries.
- Package Diagram — organize by meaningful business/technical boundaries.
- Deployment Diagram — optional if you want to practice runtime architecture.

11. RELATIONSHIP QUESTIONS FOR UML

- Customer and Order: what kind of relationship and why?
- Order and OrderItem: should item objects have an independent lifecycle from an order?
- Restaurant and Menu: can the menu exist independently in your model?
- Menu and FoodItem: consider lifecycle/ownership.

- Customer and Address: decide whether an address is an independent entity or a value-like object.
- Order and Payment: determine whether payment has its own lifecycle and identity.
- Order and Delivery: determine whether delivery is always created, optional, or independently tracked.
- Implementations and interfaces: use realization/generalization only where the subtype truly honors the abstraction.
- Services using repositories/providers: distinguish dependency from long-lived association based on your actual design.

12. DESIGN CONSTRAINTS FOR YOUR EXERCISE

- Do not put every operation into entity classes just because the data belongs to them.
- Do not create an interface for every class. Introduce abstractions where variation, substitution, or boundary isolation justifies them.
- Do not use inheritance just to reuse code; verify the LSP relationship.
- Do not make high-level business logic instantiate third-party infrastructure directly when an abstraction is appropriate.
- Do not create a giant service such as FoodDeliveryService that owns the entire application.
- Keep external provider APIs behind boundaries so provider changes do not leak into core business rules.
- Prefer explicit domain rules and valid state transitions over scattered if/else checks.
- Keep the first design simple. Refactor only where a real requirement or design smell justifies the abstraction/pattern.

13. SUGGESTED IMPLEMENTATION ORDER

- 1. Domain model and enums
- 2. User/authentication and authorization
- 3. Restaurant/menu/food management
- 4. Cart and pricing
- 5. Discount/offer rules
- 6. Checkout and order lifecycle
- 7. Payment abstraction and provider integration
- 8. Delivery partner and assignment
- 9. Notifications
- 10. Cancellation/refunds
- 11. Reviews/ratings
- 12. Complaints/admin/reporting
- 13. Persistence/infrastructure
- 14. Testing and refactoring against SOLID

14. TESTING REQUIREMENTS

- Unit-test pricing, discount eligibility, tax, delivery fee, and order-state rules independently.
- Test every allowed and forbidden order-state transition.
- Test payment success, failure, retry, and duplicate-request scenarios.
- Test cart restaurant-mixing behavior.
- Test role-based authorization.
- Test that each payment/notification/delivery implementation can be substituted through its abstraction.

- Test that adding a new implementation does not require changes to stable high-level business logic where OCP is intended.
- Use test doubles/fakes for external providers so core business logic is testable without real payment/SMS services.

15. FINAL SELF-REVIEW — BEFORE YOU CALL THE DESIGN COMPLETE

- Can I explain the responsibility of every class in one clear sentence?
- Can I list the reasons each important class could change?
- Can I add a new payment method without changing core payment orchestration?
- Can every implementation safely substitute its abstraction?
- Are interfaces focused on client needs?
- Does high-level business logic depend on abstractions rather than concrete infrastructure?
- Can I explain why each design pattern is present and what problem it solves?
- Can I remove any pattern without making the design worse? If yes, the pattern may be unnecessary.
- Can I draw the class, sequence, and state diagrams consistently with the same model?
- Can I write unit tests for the important business rules without starting the whole application?

16. PATTERN DECISION RECORD TEMPLATE

For every pattern you decide to use, document it like this:

Pattern: _____
Problem: _____
Where it occurs: _____
Why simpler code is insufficient: _____
Abstraction/participants: _____
What becomes easier: _____
Trade-off: _____
SOLID principles involved: _____

END OF SPECIFICATION